Celebal Excellence Internship 2026

Week 2 Task: E-Commerce Sales Database

Submitted By - Shubham Sharma

1. Scenario & Business Context

You are a Junior Data Analyst at ShopEase, a mid-sized e-commerce company that sells electronics, clothing, and home products across India. The management team wants to understand sales patterns, customer behavior, and product performance to make better business decisions.

You have been given access to the company's relational database containing information about customers, products, orders, and order details. Your task is to write SQL queries to extract meaningful insights from this data.

NOTE ***: I used MySQL to complete these tasks. All queries are followed by standard SQL syntax.***

2. Database Schema & Constraints

The database consists of 4 tables. Study the schema below carefully — pay attention to Primary Keys, Foreign Keys, and Constraints.

Table: customers

```
CREATE TABLE customers (
    customer_id   INT           PRIMARY KEY,
    first_name    VARCHAR(50)   NOT NULL,
    last_name     VARCHAR(50)   NOT NULL,
    email         VARCHAR(100)  UNIQUE NOT NULL,
    city          VARCHAR(50)   NOT NULL,
    state         VARCHAR(50)   NOT NULL,
    join_date     DATE          NOT NULL,
    is_premium    BOOLEAN       DEFAULT FALSE
);

-- Index for filtering by city/state
CREATE INDEX idx_customers_city ON customers(city);
CREATE INDEX idx_customers_state ON customers(state);
```

Table: products

```
CREATE TABLE products (
    product_id    INT           PRIMARY KEY,
    product_name  VARCHAR(100)  NOT NULL,
    category      VARCHAR(50)   NOT NULL,
    brand         VARCHAR(50)   NOT NULL,
    unit_price    DECIMAL(10,2) NOT NULL  CHECK (unit_price > 0),
    stock_qty     INT           NOT NULL  DEFAULT 0  CHECK (stock_qty >= 0)
);

-- Index for filtering by category
CREATE INDEX idx_products_category ON products(category);
```

Table: orders

```
CREATE TABLE orders (
    order_id      INT           PRIMARY KEY,
    customer_id   INT           NOT NULL,
    order_date    DATE          NOT NULL,
    status        VARCHAR(20)   NOT NULL  DEFAULT 'Pending'
                  CHECK (status IN ('Pending','Shipped','Delivered','Cancelled')),
```

```
    total_amount  DECIMAL(12,2) NOT NULL  CHECK (total_amount >= 0),

    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Index for date-based filtering and sorting
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);
```

Table: order_items

```
CREATE TABLE order_items (
    item_id       INT           PRIMARY KEY,
    order_id      INT           NOT NULL,
    product_id    INT           NOT NULL,
    quantity      INT           NOT NULL  CHECK (quantity > 0),
    unit_price    DECIMAL(10,2) NOT NULL  CHECK (unit_price > 0),
    discount_pct  DECIMAL(5,2)  DEFAULT 0 CHECK (discount_pct BETWEEN 0 AND 100),

    FOREIGN KEY (order_id)   REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Entity Relationships

```
  customers  ──(1:N)──►  orders
  orders     ──(1:N)──►  order_items
  products   ──(1:N)──►  order_items

  customers.customer_id  ◄────FK────  orders.customer_id
  orders.order_id        ◄────FK────  order_items.order_id
  products.product_id    ◄────FK────  order_items.product_id
```

3. Sample Data

Below is sample data to help you understand the tables. Use the provided INSERT statements to load data into your database.

customers (sample rows)

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	TRUE
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	FALSE
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	TRUE
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	FALSE
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	TRUE
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	FALSE
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	TRUE
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	FALSE

products (sample rows)

product_id	product_name	category	brand	unit_price	stock_qty
201	Wireless Earbuds	Electronics	BoAt	1499.0	250
202	Cotton T-Shirt	Clothing	Levi's	799.0	500
203	Smart Watch	Electronics	Noise	2999.0	150
204	Running Shoes	Clothing	Nike	4599.0	120
205	Bluetooth Speaker	Electronics	JBL	3499.0	200
206	Bedsheet Set	Home	Spaces	1299.0	300
207	Laptop Stand	Electronics	AmazonBasics	899.0	180
208	Cushion Covers (Set)	Home	HomeCenter	599.0	400

orders (sample rows)

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.0
1002	102	2024-08-03	Delivered	799.0
1003	103	2024-08-05	Shipped	7498.0
1004	101	2024-08-10	Delivered	3499.0
1005	104	2024-08-12	Cancelled	2999.0
1006	105	2024-08-15	Delivered	5898.0
1007	106	2024-08-18	Pending	1299.0
1008	103	2024-08-20	Delivered	899.0
1009	107	2024-08-25	Shipped	6098.0
1010	108	2024-08-28	Delivered	1598.0

order_items (sample rows)

item_id	order_id	product_id	quantity	unit_price	discount_pct
5001	1001	201	2	1499.0	0
5002	1001	207	1	899.0	10
5003	1002	202	1	799.0	0
5004	1003	203	1	2999.0	0

5005	1003	204	1	4599.0	5
5006	1004	205	1	3499.0	0
5007	1005	203	1	2999.0	0
5008	1006	201	1	1499.0	10
5009	1006	204	1	4599.0	5
5010	1007	206	1	1299.0	0
5011	1008	207	1	899.0	0
5012	1009	205	1	3499.0	0
5013	1009	208	2	599.0	15
5014	1010	206	1	1299.0	0
5015	1010	208	1	599.0	0

Data Loading — INSERT Statements

Copy and execute the following SQL to load sample data into your database:

```
-- ========== INSERT: customers ==========
INSERT INTO customers VALUES
(101, 'Aarav',  'Sharma', 'aarav.s@email.com',  'Mumbai',    'Maharashtra',
'2024-01-15', TRUE),
(102, 'Priya',  'Patel',  'priya.p@email.com',  'Ahmedabad', 'Gujarat',
'2024-02-20', FALSE),
(103, 'Rohan',  'Gupta',  'rohan.g@email.com',  'Delhi',     'Delhi',
'2024-03-10', TRUE),
(104, 'Sneha',  'Reddy',  'sneha.r@email.com',  'Hyderabad', 'Telangana',
'2024-04-05', FALSE),
(105, 'Vikram', 'Singh',  'vikram.s@email.com', 'Jaipur',    'Rajasthan',
'2024-05-12', TRUE),
(106, 'Ananya', 'Iyer',   'ananya.i@email.com', 'Chennai',   'Tamil Nadu',
'2024-06-18', FALSE),
(107, 'Karan',  'Mehta',  'karan.m@email.com',  'Pune',      'Maharashtra',
'2024-07-22', TRUE),
(108, 'Divya',  'Nair',   'divya.n@email.com',  'Kochi',     'Kerala',
'2024-08-30', FALSE);

-- ========== INSERT: products ==========
INSERT INTO products VALUES
(201, 'Wireless Earbuds',     'Electronics', 'BoAt',          1499.00, 250),
(202, 'Cotton T-Shirt',       'Clothing',    'Levis',         799.00,  500),
(203, 'Smart Watch',          'Electronics', 'Noise',         2999.00, 150),
(204, 'Running Shoes',        'Clothing',    'Nike',          4599.00, 120),
(205, 'Bluetooth Speaker',    'Electronics', 'JBL',           3499.00, 200),
(206, 'Bedsheet Set',         'Home',        'Spaces',        1299.00, 300),
(207, 'Laptop Stand',         'Electronics', 'AmazonBasics',  899.00,  180),
(208, 'Cushion Covers (Set)', 'Home',        'HomeCenter',    599.00,  400);

-- ========== INSERT: orders ==========
INSERT INTO orders VALUES
(1001, 101, '2024-08-01', 'Delivered',  4498.00),
(1002, 102, '2024-08-03', 'Delivered',  799.00),
(1003, 103, '2024-08-05', 'Shipped',    7498.00),
(1004, 101, '2024-08-10', 'Delivered',  3499.00),
(1005, 104, '2024-08-12', 'Cancelled',  2999.00),
(1006, 105, '2024-08-15', 'Delivered',  5898.00),
(1007, 106, '2024-08-18', 'Pending',    1299.00),
(1008, 103, '2024-08-20', 'Delivered',  899.00),
(1009, 107, '2024-08-25', 'Shipped',    6098.00),
(1010, 108, '2024-08-28', 'Delivered',  1598.00);
```

```
-- ========== INSERT: order_items ==========
INSERT INTO order_items VALUES
(5001, 1001, 201, 2, 1499.00, 0),
(5002, 1001, 207, 1, 899.00,  10),
(5003, 1002, 202, 1, 799.00,  0),
(5004, 1003, 203, 1, 2999.00, 0),
(5005, 1003, 204, 1, 4599.00, 5),
(5006, 1004, 205, 1, 3499.00, 0),
(5007, 1005, 203, 1, 2999.00, 0),
(5008, 1006, 201, 1, 1499.00, 10),
(5009, 1006, 204, 1, 4599.00, 5),
(5010, 1007, 206, 1, 1299.00, 0),
(5011, 1008, 207, 1, 899.00,  0),
(5012, 1009, 205, 1, 3499.00, 0),
(5013, 1009, 208, 2, 599.00,  15),
(5014, 1010, 206, 1, 1299.00, 0),
(5015, 1010, 208, 1, 599.00,  0);
```

4. My Answering Format Flow

4.1-What is The Question.

4.2 Answer As Query or Theory Answer.

4.3 Screenshot of the Result of the Query.

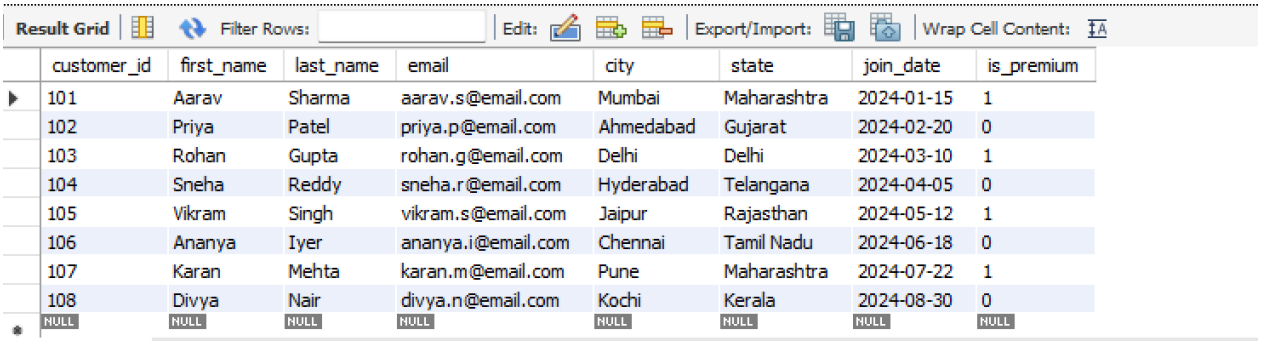
4.4 Explanation.

5. Questions & Their Answers.

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

Ans - SELECT * FROM customers;

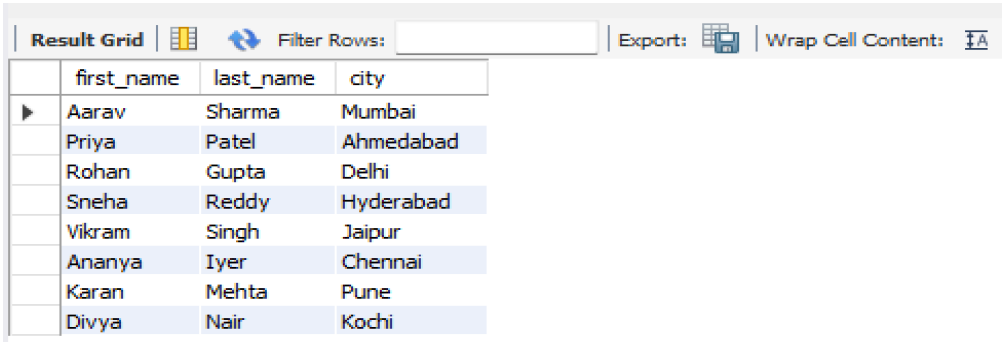


customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation - This query retrieves all columns and all rows from the customers table. It provides a complete view of customer information including customer ID, name, email, location, join date, and premium membership status.

Q2. Retrieve only the first_name, last_name, and city of all customers.

Ans - SELECT first_name,last_name,city FROM customers;

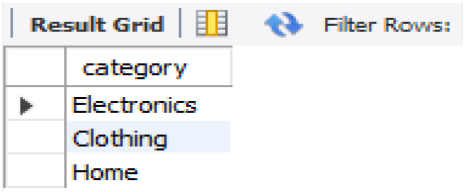


first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

Explanation - This query selects only the first name, last name, and city columns from the customers table.

Q3. List all unique categories available in the products table.

Ans - **SELECT DISTINCT category FROM products;**



Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Ans - Each table's Primary Key:

1. customers → customer_id
2. products → product_id
3. orders → order_id
4. order_items → item_id

A Primary Key uniquely identifies every record in a table.

Properties:

- Must be unique
- Cannot contain NULL values
- Ensures data integrity
- Prevents duplicate records

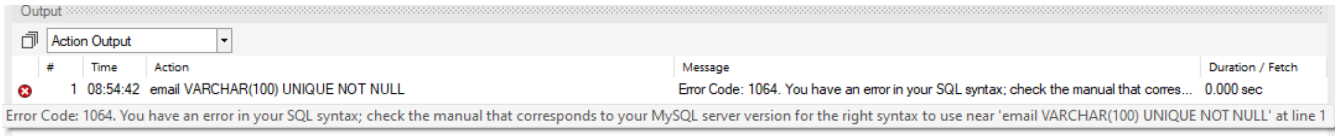
Unique – No two rows can have the same primary key value. This ensures that every record can be identified individually.

NOT NULL – A primary key cannot contain NULL values because every record must have a valid identifier.

Example: In the customers table, customer_id is the primary key. If two customers had the same customer_id, the database would not be able to distinguish between them. Similarly, if customer_id were NULL, the customer record could not be uniquely identified.Therefore, primary keys help maintain data integrity and ensure that each record in a table is uniquely identifiable.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Ans - email VARCHAR(100) UNIQUE NOT NULL.



The `email` column has:

- NOT NULL constraint
- UNIQUE constraint

Explanation :

`NOT NULL` ensures every customer has an email address.

`UNIQUE` ensures duplicate email addresses cannot exist in the database.

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

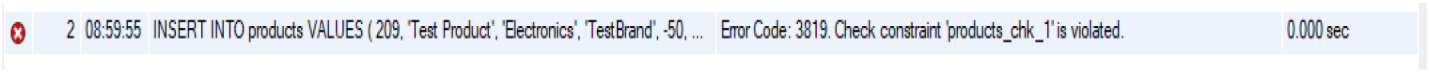
Ans - INSERT INTO products VALUES (209, 'Test Product', 'Electronics', 'TestBrand', -50, 100);

Explanation

-> The insertion fails because the products table contains the following CHECK constraint:

-> CHECK (unit_price > 0)

-> Since -50 is less than 0, the database rejects the record

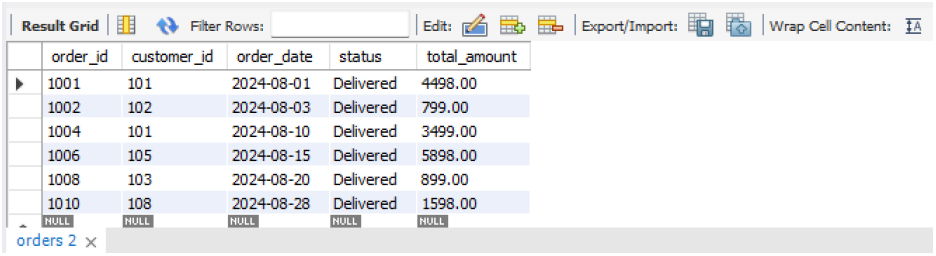


⊗	2	08:59:55	INSERT INTO products VALUES (209, 'Test Product', 'Electronics', 'TestBrand', -50, ...	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

Ans - SELECT * FROM orders WHERE status = 'Delivered';



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

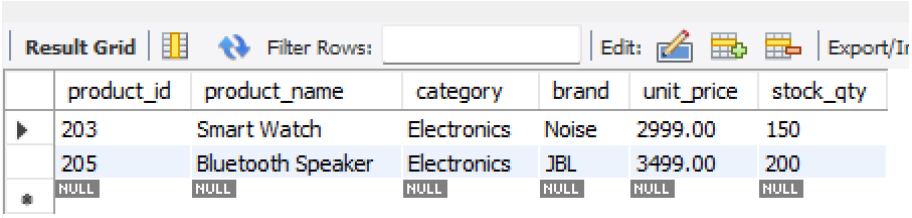
orders 2

Explanation :

-> This query filters records and returns only customers whose city is Mumbai.

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

Ans - SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;



Result Grid | Filter Rows: | Edit: | Export/Ir

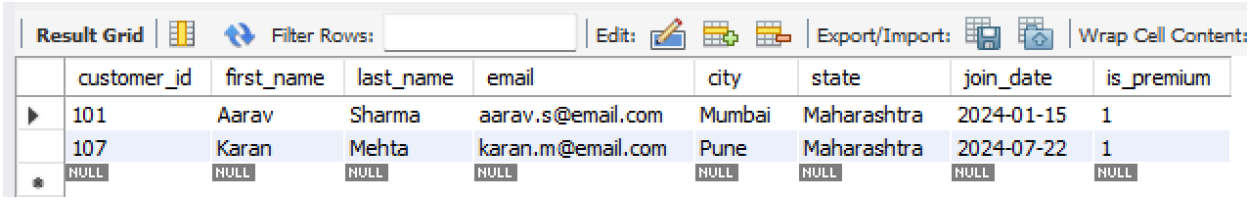
product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

Explanation :

->Returns all products whose selling price exceeds ₹2000.

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'

Ans - SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01' AND state = 'Maharashtra';



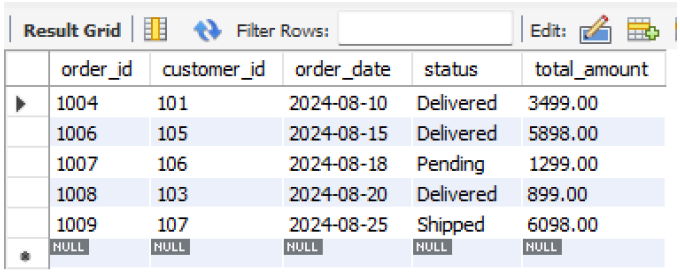
customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation :

-> Displays all orders placed after the specified date.

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Ans - SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled'



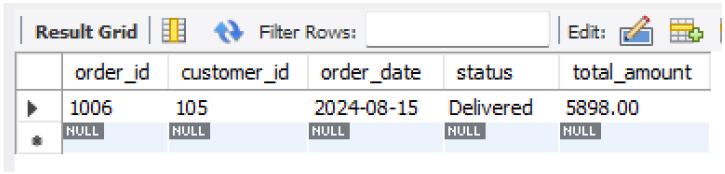
order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Explanation :

-> Returns all customers enrolled in the premium membership program**.**

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

Ans - SELECT * FROM orders WHERE order_date = '2024-08-15';



order_id	customer_id	order_date	status	total_amount
1006	105	2024-08-15	Delivered	5898.00
NULL	NULL	NULL	NULL	NULL

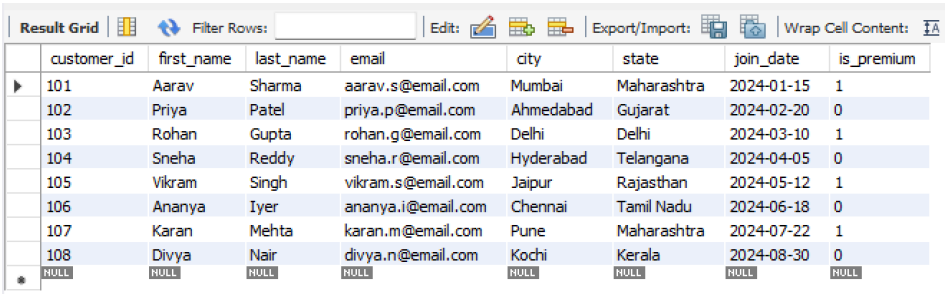
Benefits

- **Faster search operations**
- **Faster filtering using WHERE**
- **Improved JOIN performance**
- **Reduced table scanning**

Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Ans - SELECT * FROM customers WHERE YEAR(join_date) = 2024;

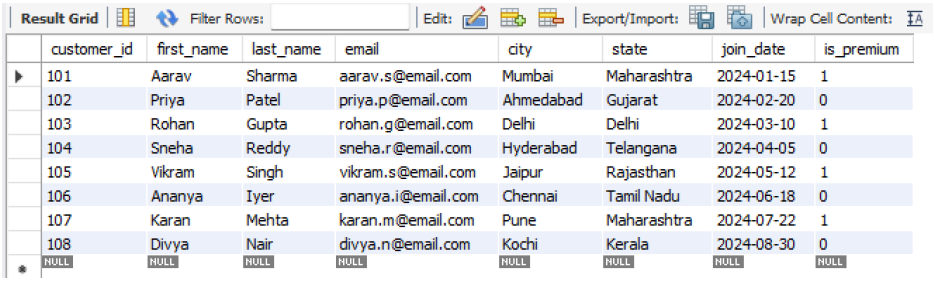
Why the index is NOT used: The YEAR() function wraps the indexed column. The database engine cannot use the index because it does not store pre-computed YEAR() values, it must evaluate the function for every row, resulting in a full table scan.



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01'



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Explanation :

-> The first query can use an index directly.

-> The second query applies a function to the indexed column, forcing MySQL to scan all rows.

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

Ans - SELECT COUNT(*) AS total_orders FROM orders;

Result Grid

	total_orders
▶	11

Explanation :

-> COUNT(*) returns the total number of rows.

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

Ans - SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered';

Result Grid

	total_revenue
▶	17191.00

Explanation :

-> SUM() aggregates inventory quantities across all products.

Q15. Calculate the average unit_price of products in each category.

Ans - SELECT category, ROUND(AVG(unit_price), 2) AS avg_unit_price FROM products GROUP BY category;

Result Grid | Filter Rows:

	category	avg_unit_price
▶	Electronics	2224.00
	Clothing	2699.00
	Home	949.00

Explanation :

-> AVG() calculates the mean product price.

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

Ans - SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_revenue FROM orders GROUP BY status ORDER BY total_revenue DESC;

Result Grid | Filter Rows:

status	order_count	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	2	2897.00

Explanation : MAX() returns the highest value in the column

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

Ans - SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest FROM products GROUP BY category;

Result Grid | Filter Rows:

category	most_expensive	cheapest
Electronics	3499.00	899.00
Clothing	4599.00	799.00
Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

Ans - SELECT category, ROUND(AVG(unit_price), 2) AS avg_unit_price FROM products GROUP BY category HAVING AVG(unit_price) > 2000;

Result Grid | Filter Rows:

category	avg_unit_price
Electronics	2224.00
Clothing	2699.00

Explanation :

-> GROUP BY creates category-wise product counts.

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

Ans - SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id;

Result Grid | Filter Rows: | Export:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Result 19

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

Ans - SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.total_amount FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

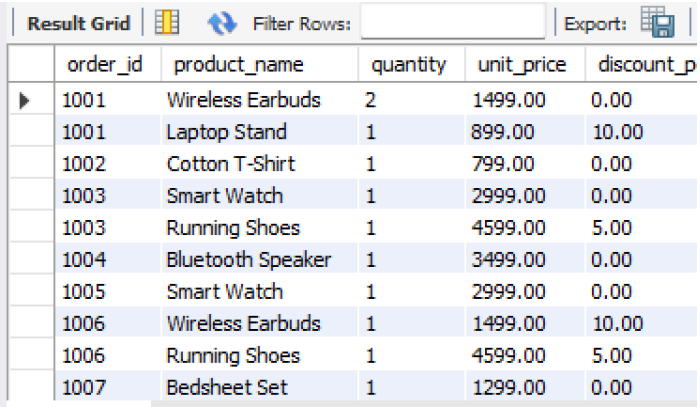
customer_id	first_name	last_name	order_id	order_date	total_amount
101	Aarav	Sharma	1001	2024-08-01	4498.00
101	Aarav	Sharma	1004	2024-08-10	3499.00
102	Priya	Patel	1002	2024-08-03	799.00
103	Rohan	Gupta	1003	2024-08-05	7498.00
103	Rohan	Gupta	1008	2024-08-20	899.00
104	Sneha	Reddy	1005	2024-08-12	2999.00
105	Vikram	Singh	1006	2024-08-15	5898.00
106	Ananya	Iyer	1007	2024-08-18	1299.00
107	Karan	Mehta	1009	2024-08-25	6098.00
108	Divya	Nair	1010	2024-08-28	1598.00

Result 20

Q21. Write a query using JOINs across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

Ans - SELECT oi.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct FROM order_items oi JOIN orders o ON oi.order_id = o.order_id JOIN products p ON oi.product_id = p.product_id ORDER BY

oi.order_id;



Result Grid | Filter Rows: | Export:

order_id	product_name	quantity	unit_price	discount_p
1001	Wireless Earbuds	2	1499.00	0.00
1001	Laptop Stand	1	899.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1005	Smart Watch	1	2999.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1006	Running Shoes	1	4599.00	5.00
1007	Bedsheet Set	1	1299.00	0.00

Explanation : INNER JOIN returns only those records where matching customer IDs exist in both tables.

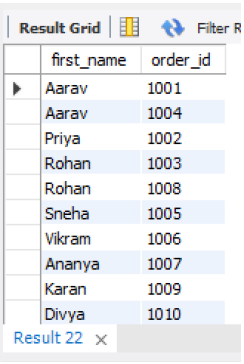
- **Customers without orders are excluded.**
- **Orders without valid customers are excluded.**

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Ans - Difference Between LEFT and RIGHT JOIN

	LEFT JOIN	RIGHT JOIN
1.	Returns all rows from left table	Returns all rows from right table
2.	Missing matches on right become NULL	Missing matches on left become NULL

SELECT c.first_name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;



Result Grid | Filter R

first_name	order_id
Aarav	1001
Aarav	1004
Priya	1002
Rohan	1003
Rohan	1008
Sneha	1005
Vikram	1006
Ananya	1007
Karan	1009
Divya	1010

Result 22

SELECT c.first_name, o.order_id FROM customers c RIGHT JOIN orders o ON c.customer_id =

Result Grid | Filter Ro

first_name	order_id
Aarav	1001
Aarav	1004
Priya	1002
Rohan	1003
Rohan	1008
Sneha	1005
Vikram	1006
Ananya	1007
Karan	1009
Divya	1010

Result 23 ×

o.customer_id;

Explanation : LEFT JOIN returns:

- All records from the left table (customers)
- Matching records from the right table (orders)
- NULL values where no matching order exists

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Ans - The three Foreign Key relationships in the schema:

• orders.customer_id ⟶ customers.customer_id
• order_items.order_id ⟶ orders.order_id
• order_items.product_id ⟶ products.product_id if we try to insert an order with customer_id = 999
using- **INSERT INTO orders VALUES (1011, 999, '2024-08-31', 'Pending', 500.00);**

11 19:20:46 INSERT INTO orders VALUES (1011, 999, '2024-08-31', 'Pending', 500.00) Error Code: 1062. Duplicate entry '1011' for key 'orders.PRIMARY'

Explanation :

-> RIGHT JOIN returns:

- **All rows from orders table**
- **Matching rows from customers table**
- **NULL values for unmatched customer record**

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Ans - Foreign Key Relationships :

Child Table	Foreign Key	Parent Table
1.orders	customer_id	customers
2.order_items	order_id	orders
3.order_items	product_id	products

SELECT product_name, unit_price, CASE WHEN unit_price < 1000 THEN 'Budget' WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range' ELSE 'Premium' END AS price_tier FROM products;

Result Grid | Filter Rows:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

Result 24 ×

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Ans - SELECT SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_orders, SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS not_delivered_orders FROM orders;

Result Grid | Filter Rows:

delivered_orders	not_delivered_orders
6	4

Explanation : The CASE statement works like an IF-ELSE condition in SQL.

Orders are categorized into: 1. High Value , 2. Medium Value , 3. Low Value .

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Ans - ACID is a set of four properties that guarantee reliable database transactions:

A — Atomicity

All operations in a transaction are treated as a single unit. Either ALL succeed or ALL are rolled back. Example: In a bank transfer, debiting Account A and crediting Account B must both happen, if the credit fails, the debit is also undone. No money is lost in the middle.

C — Consistency

A transaction brings the database from one valid state to another. All defined rules (constraints, foreign keys, check conditions) must hold before and after. Example: A bank transfer must not result in a negative balance if the CHECK constraint prohibits it.

I — Isolation

Concurrent transactions execute as if they were serial, they do not see each other's intermediate states. Example: If two people simultaneously try to book the last seat on a flight, isolation ensures only one succeeds; the other sees the updated seat count.

D — Durability

Once a transaction is committed, its changes are permanent even if the system crashes immediately afterward. The database uses write-ahead logs (WAL) and disk flushes to guarantee this .

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)

Ans - BEGIN; -- Step 1: Insert new order

INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (1011, 102, CURRENT_DATE, 'Pending', 1598.00);

order_id	customer_id	order_date	status	total_amount
1011	102	2026-05-31	Pending	1598.00
NULL	NULL	NULL	NULL	NULL

2. Insert two order items for that order

-- Step 2a: First order item (Bedsheet Set, qty 1)

INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct) VALUES (5016, 1011, 206, 1, 1299.00, 0);

item_id	order_id	product_id	quantity	unit_price	discount_pct
5016	1011	206	1	1299.00	0.00
NULL	NULL	NULL	NULL	NULL	NULL

-- **Step 2b:** Second order item (Cushion Covers, qty 1)
INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct) VALUES (5017, 1011, 208, 1, 599.00, 0);

item_id	order_id	product_id	quantity	unit_price	discount_pct
5016	1011	206	1	1299.00	0.00
NULL	NULL	NULL	NULL	NULL	NULL

3. Update the stock_qty of the purchased products

-- **Step 3a:** Reduce stock for Bedsheet Set
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206;

product_id	product_name	stock_qty
206	Bedsheet Set	299
208	Cushion Covers (Set)	400
NULL	NULL	NULL

-- **Step 3b:** Reduce stock for Cushion Covers
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208;

product_id	product_name	stock_qty
206	Bedsheet Set	299
208	Cushion Covers (Set)	399
NULL	NULL	NULL

COMMIT;

-- If any step fails: ROLLBACK; (undoes all changes atomically)

This transaction is atomic: if any of the 5 statements fails (e.g., invalid product_id,c stock goes negative), issuing ROLLBACK reverts every change made so far. If all succeed, COMMIT makes them permanent.

Conclusion

This assignment demonstrated the application of SQL fundamentals to perform data exploration, filtering, aggregation, joins, transaction management, and business-oriented analysis on a sales dataset.

Throughout the assignment, various SQL concepts were implemented, including:

- **SELECT statements for data retrieval**

- **WHERE clause for filtering records**
- **Aggregate functions (SUM, COUNT, AVG, MIN, MAX)**
- **GROUP BY for summarized reporting**
- **ORDER BY and LIMIT for ranking and top-N analysis**
- **INNER JOIN and LEFT JOIN for combining data across multiple tables**
- **Primary Keys and Foreign Keys for maintaining data integrity**
- **CASE statements for conditional categorization**
- **Transaction Management using START TRANSACTION, COMMIT, and ROLLBACK**
- **Duplicate record identification and data validation techniques**

The analysis provided valuable business insights such as:

- **Identifying top-performing products**
- **Understanding customer purchasing behavior**
- **Analyzing sales trends**
- **Monitoring order statuses**
- **Detecting duplicate records**
- **Evaluating inventory availability**
- **Generating summarized reports for business decision-making**

This assignment helped strengthen practical SQL skills and provided hands-on experience with real-world database operations commonly used by Data Analysts, Data Engineers, and Business Intelligence professionals.

Key Learnings

1. **Designing and working with relational databases.**
2. **Writing efficient SQL queries for business requirements.**
3. **Using aggregate functions for reporting and analytics.**
4. **Applying joins to combine data from multiple tables.**
5. **Ensuring data integrity using constraints and keys.**
6. **Handling transactions to maintain database consistency.**
7. **Extracting meaningful insights from raw business data.**
8. **Following industry-standard SQL development practices.**

Outcome

The assignment successfully achieved its objective of performing SQL-based data analysis using filtering, aggregation, joins, transactions, and business queries. The generated insights demonstrate how SQL can be used to transform raw transactional data into meaningful business information for decision-making.

----------END----------